

DLD Assignment #3

Sa10303

Q1:-

x_3	x_2	x_1	x_0	B_3	B_2	B_1	B_0	Decimal equivalent
0	0	1	1	0	0	0	0	0
0	1	0	0	0	0	0	1	1
0	1	0	1	0	0	1	0	2
0	1	1	0	0	0	1	1	3
0	1	1	1	0	1	0	0	4
1	0	0	0	0	1	0	1	5
1	0	0	1	0	1	1	0	6
1	0	1	0	0	1	1	1	7
1	0	1	1	1	0	0	0	8
1	1	0	0	1	0	0	1	9

equation for B_0 :-

$x_3 x_2 \backslash x_1 x_0$	00	01	11	10
00	X	X	0	X
01	1	0	0	1
11	1	X	X	X
10	1	0	0	1

equation = x_0'

equation for B_1 :-

$x_3 x_2 \backslash x_1 x_0$	00	01	11	10
00	X	X	0	X
01	0	1	0	1
11	0	X	X	X
10	0	1	0	1

equation:-

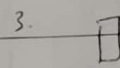
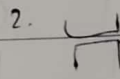
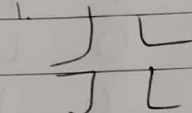
$\overline{x_1} x_0 + x_1 \overline{x_0}$

or

$x_0 \oplus x_1$

K-map of B_2 :-

$x_3 x_2 \backslash x_1 x_0$	00	01	11	10
00	1	1	0	1
01	0	0	1	0
11	0	1	1	1
10	1	1	0	1



equation:-

$$\overline{x_2} \overline{x_0} + \overline{x_2} x_1 + x_0 x_1 x_2$$

K-map of B_3 :-

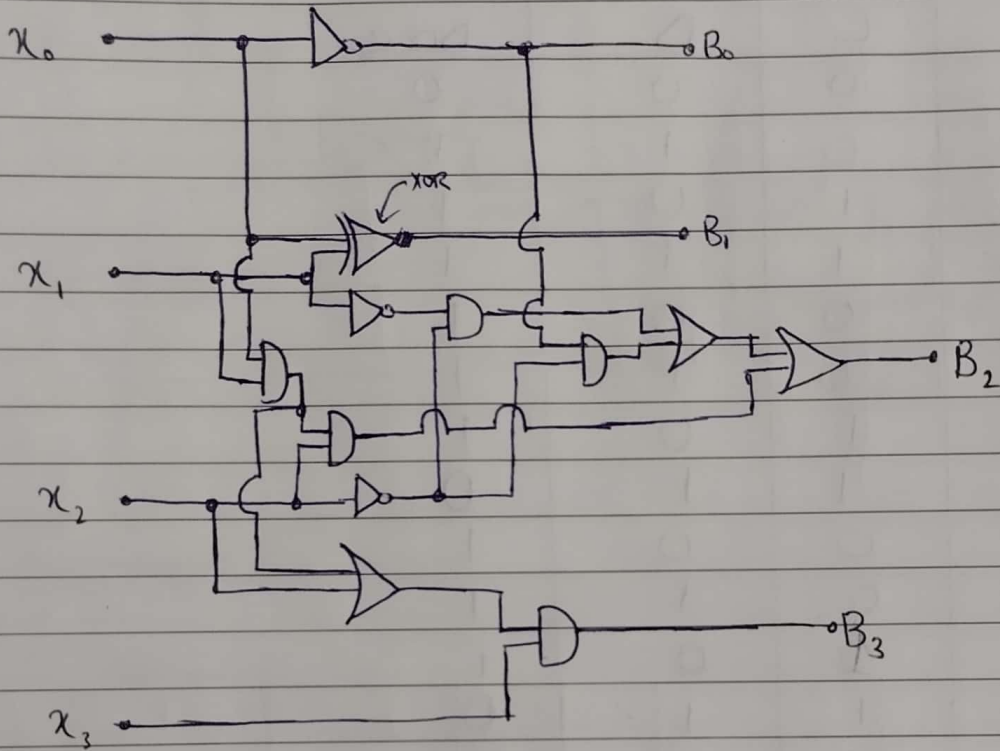
$x_3 x_2 \backslash x_1 x_0$	00	01	11	10
00	1	1	0	1
01	0	0	0	0
11	1	1	1	1
10	0	0	1	0

equation:-

$$x_3 x_2 + x_0 x_1 x_3 = x_3 (x_2 + x_0 x_1)$$



Circuit:-



Question #2:-

First we will make the truth table:-

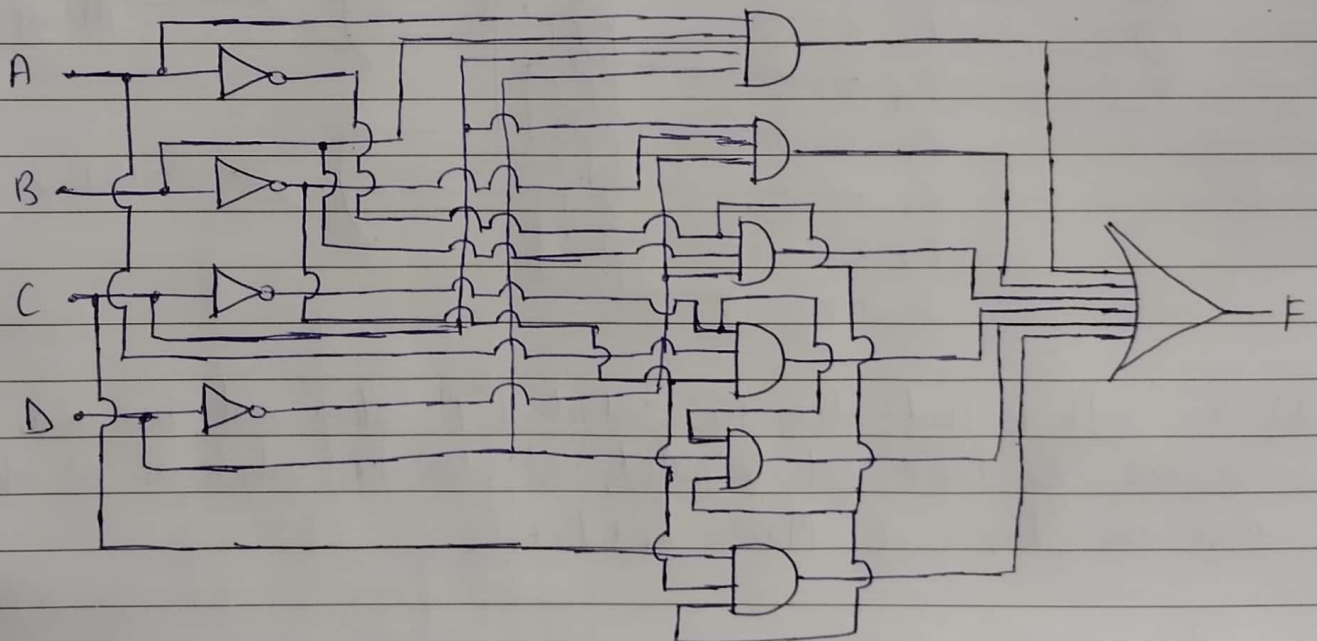
A	B	C	D	Door
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

Q2:- K-maps:-

AB \ CD	00	01	11	10
00	0	1	1	1
01	1	1	0	1
11	0	0	1	0
10	1	1	0	1

$$F = \overline{B}\overline{C}\overline{D} + \overline{A}B\overline{D} + ABCD + A\overline{B}\overline{C} + \overline{A}\overline{C}D + \overline{A}\overline{B}C$$

Circuit implementation:-



QUESTION # 3:-

a)

Truth table:-

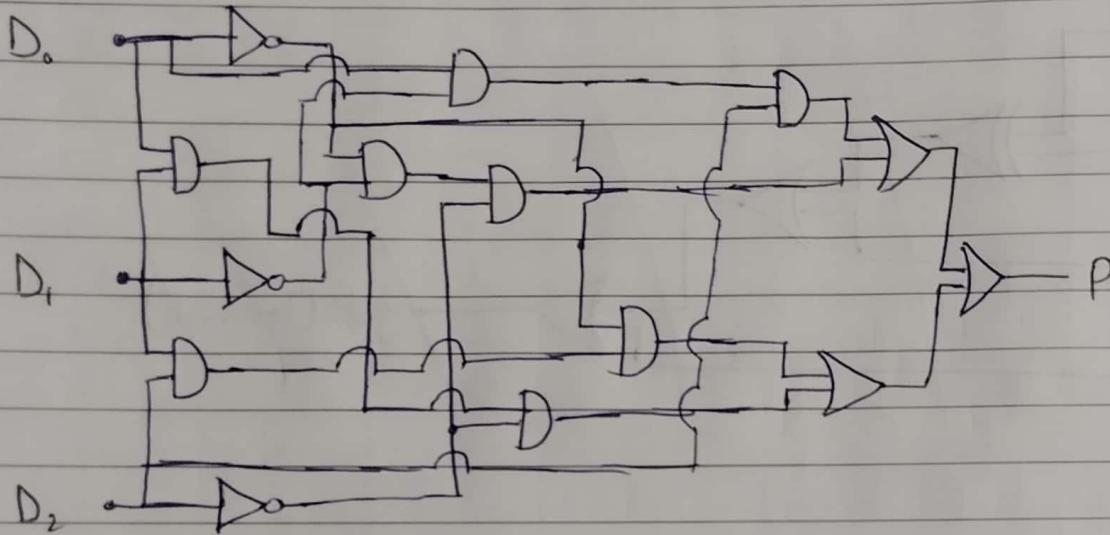
D_2	D_1	D_0	P
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

K-maps:-

$D_2 D_1 \backslash D_0$		0	1
00		1	0
01		0	1
11		1	0
10		0	1

$$\bar{D}_0 \bar{D}_1 \bar{D}_2 + D_0 D_1 \bar{D}_2 + \bar{D}_0 D_1 D_2 + D_0 \bar{D}_1 D_2$$

Circuit diagram:-

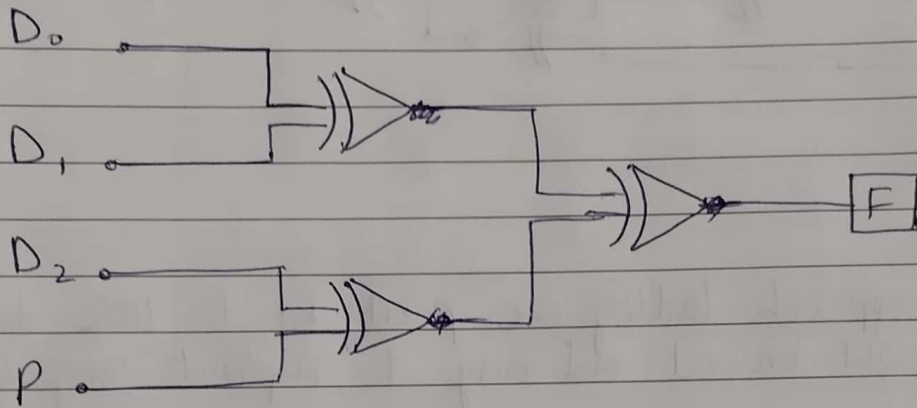


b)

D_2	D_1	D_0	P	F	Total number of 1's
0	0	0	1	0	1
0	0	1	0	0	1
0	1	0	0	0	1
0	1	1	1	0	3
1	0	0	0	0	1
1	0	1	1	0	3
1	1	0	1	0	3
1	1	1	0	0	3
0	0	0	0	1	
0	0	1	1	1	
0	1	0	1	1	
0	1	1	0	1	
1	0	0	1	1	
1	0	1	0	1	
1	1	0	0	1	
1	1	1	1	1	

if exactly 2 should be zero, or there is no error in the

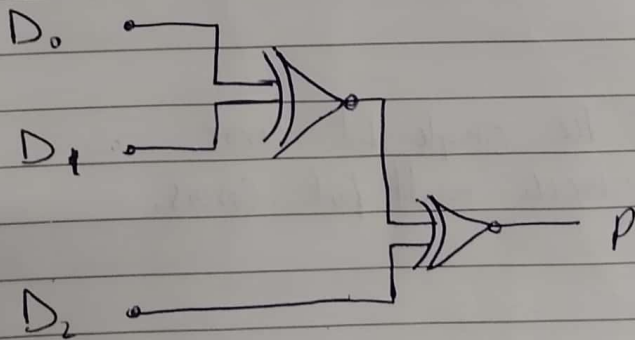
logic diagram:- equation = $F = D_2 \oplus D_1 \oplus D_0 \oplus P$



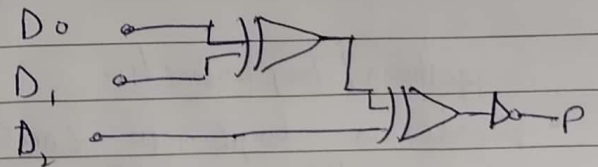
c) Provide NXOR implementation of Part A:-

$$P = D_2 \oplus D_1 \oplus D_0$$

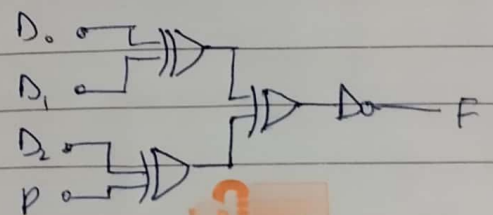
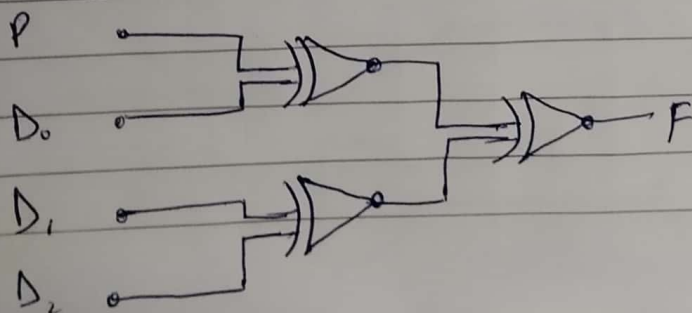
$$F = D_2 \oplus D_1 \oplus D_0 \oplus P$$

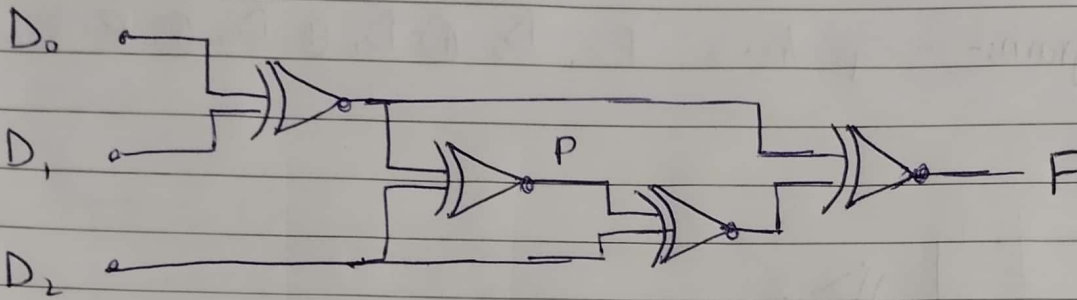


XOR:-



XOR:-





e) advantages:-

- Odd parity approach (adding one parity bit to three data bits) is straight forward and easy to implement using basic logic gates.
- Only a single bit is added if the number of highs is ~~even~~ odd so ~~the~~ ~~all other~~ ~~one~~ that it minimizes bandwidth or storage consumption.

Disadvantages:-

- if two bits are flipped, the number of 1s will still be ~~an~~ odd bit system will not detect an error.
- No error correction.

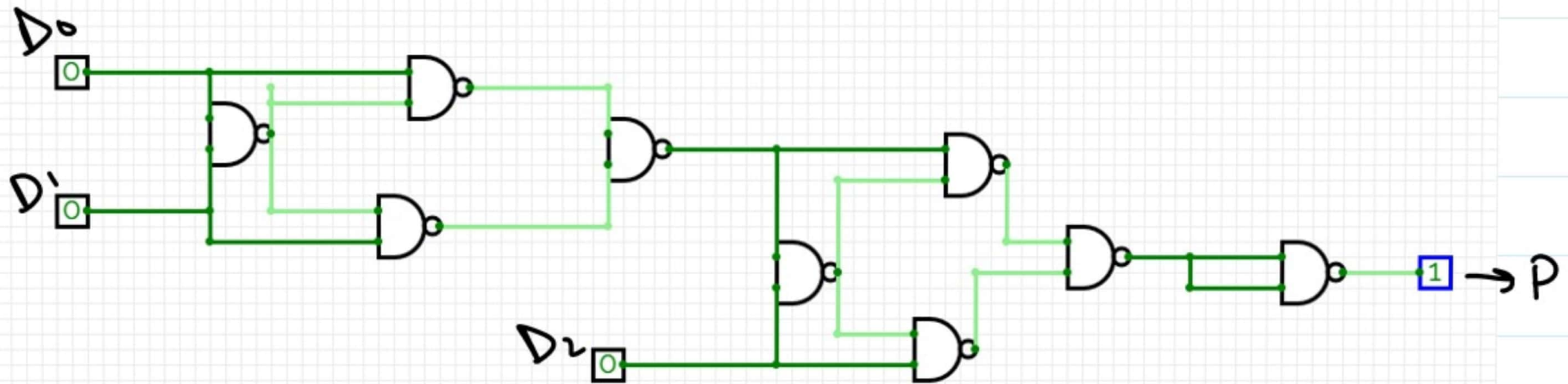
Suggestions/Improvements:-

- provide a logic to correct the single-bit error.
- Using multiple parity bits, check multiple errors.

d) ∴ screen shots attached!

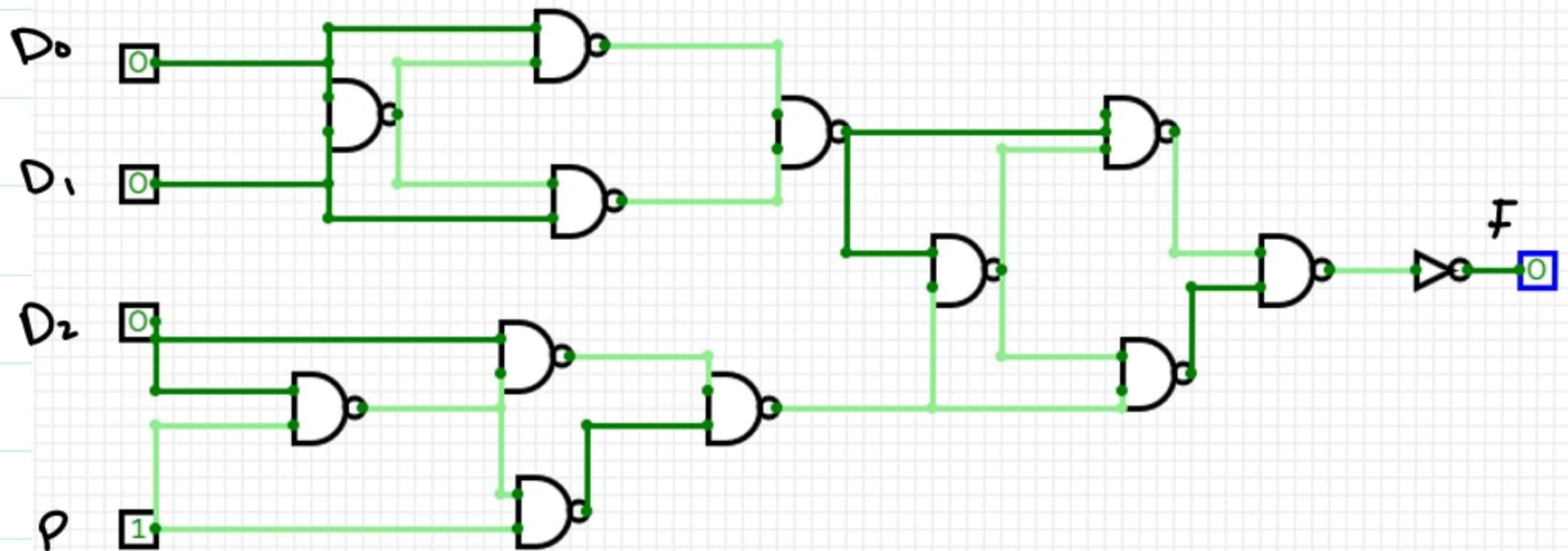


Testcase 1:-



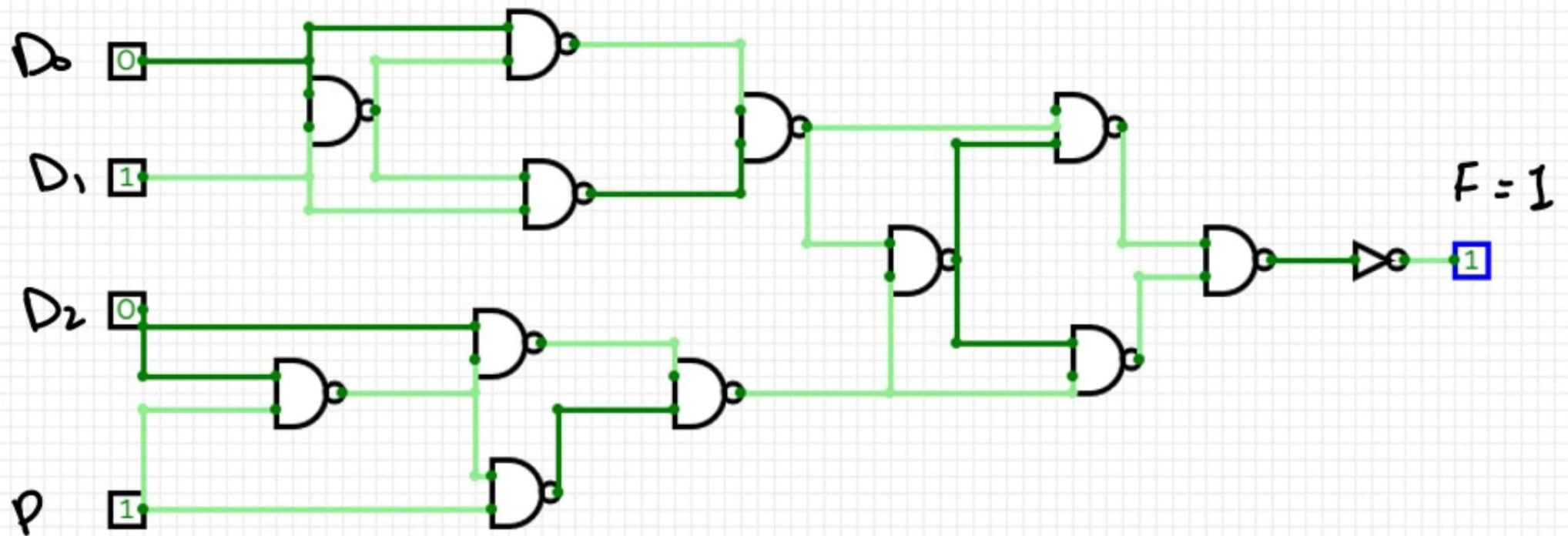
NAND IMPLEMENTATION OF $P = \overline{D_0 \oplus D_1 \oplus D_2}$

Testcase II :-



when there is no error.

Test case III:- There is an error.



$\therefore P$ should be zero.

Q41-

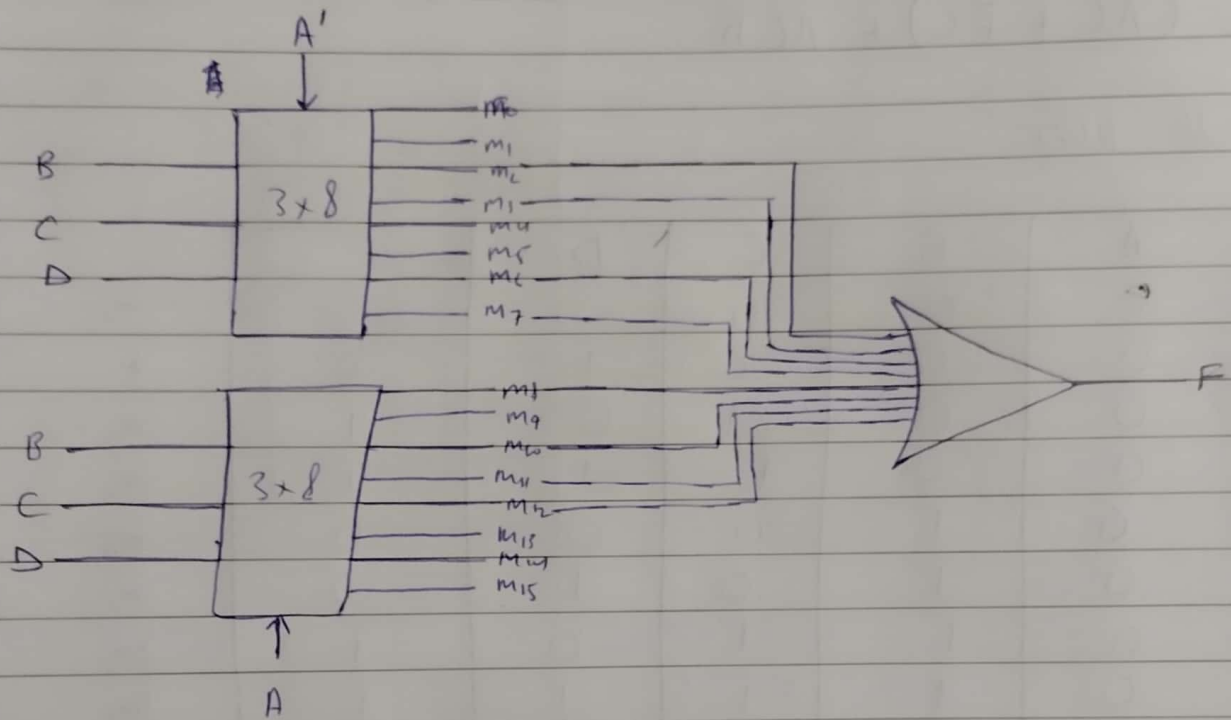
a) Boolean representation:-

$$= (\bar{A}C + \bar{B}C) + AC\bar{D}$$

Truth table-

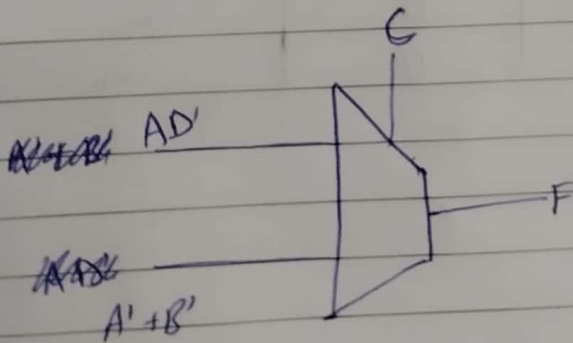
A	B	C	D	F	
0	0	0	0	0	m_0
0	0	0	1	0	m_1
0	0	1	0	1	m_2
0	0	1	1	1	m_3
0	1	0	0	0	m_4
0	1	0	1	0	m_5
0	1	1	0	1	m_6
0	1	1	1	1	m_7
1	0	0	0	1	m_8
1	0	0	1	0	m_9
1	0	1	0	1	m_{10}
1	0	1	1	1	m_{11}
1	1	0	0	1	m_{12}
1	1	0	1	0	m_{13}
1	1	1	0	0	m_{14}
1	1	1	1	0	m_{15}

b) $\Sigma(2, 3, 6, 7, 8, 10, 11, 12)$

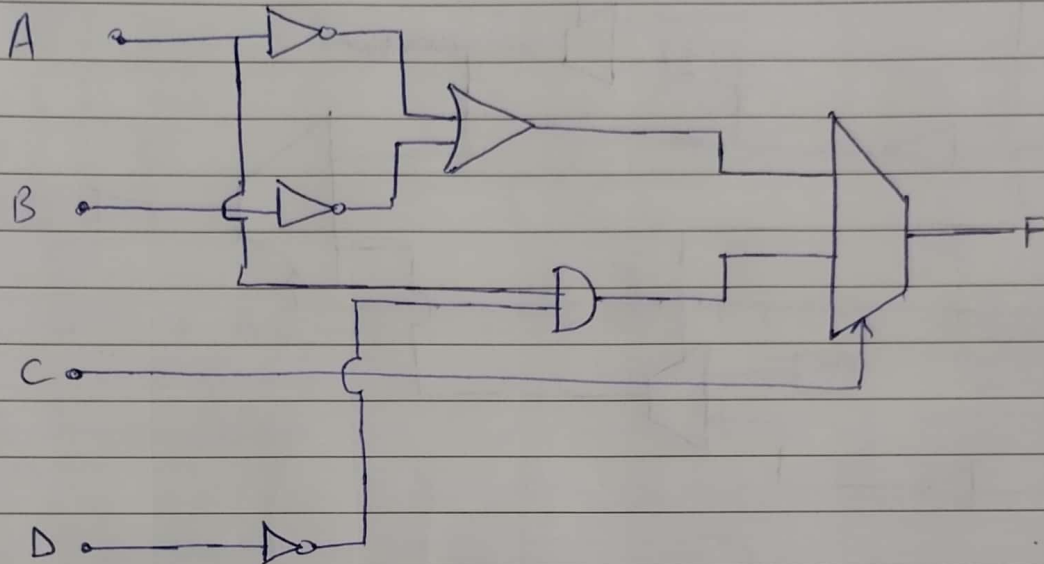


c) equation = $(A'C + B'C) + AC'D'$
 $C(B' + A') + C'AD'$

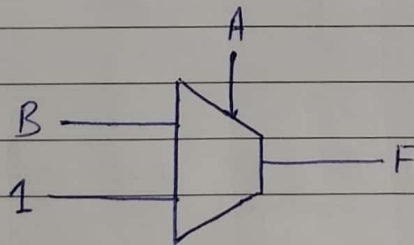
if $C=1$ $F = A' + B'$
 $C=0$ $F = AD'$



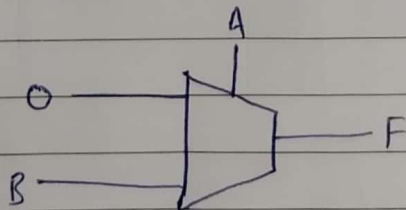
Q1-



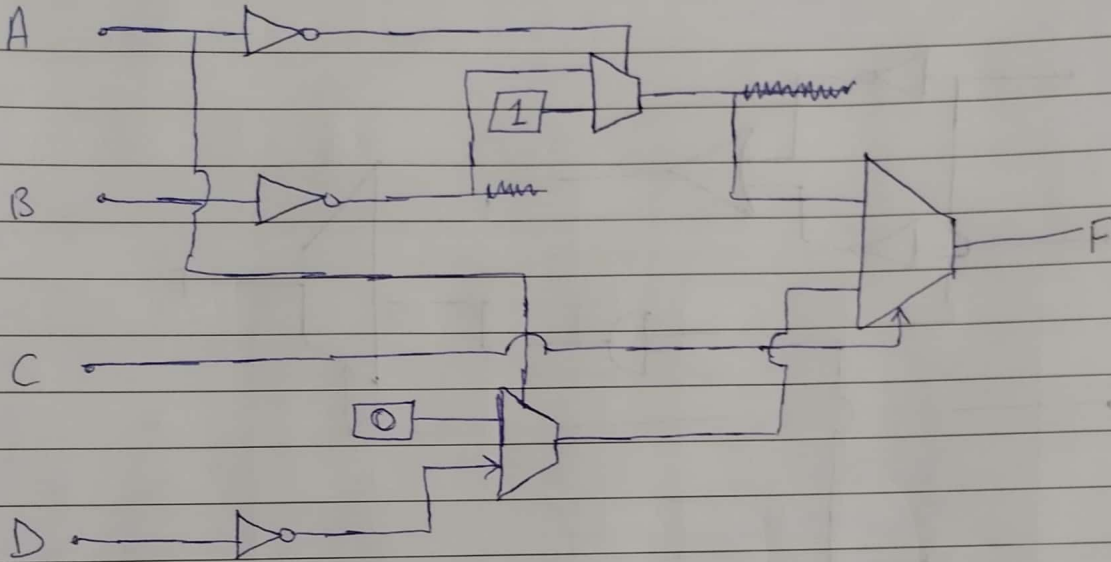
d) OR gate by mux:-



AND gate by mux:-



Circuit becomes:-



Q5:-

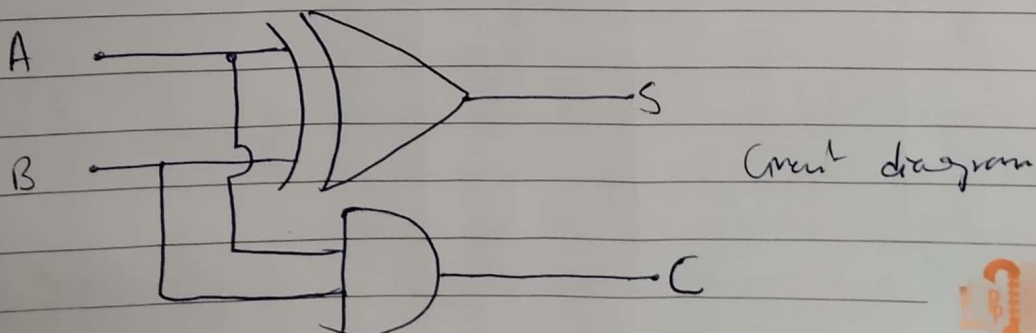
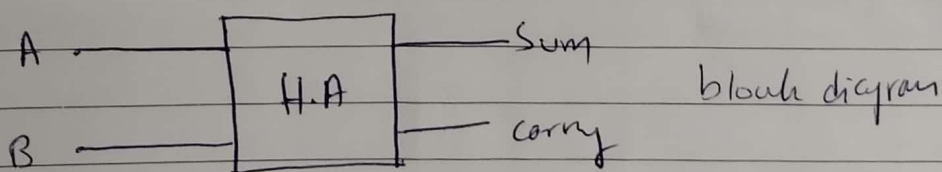
a) 2-bit by 4-bit multiplier:-

2-bit number = ~~A₂~~ A₁ A₀4-bit number = B₃ B₂ B₁ B₀Product = P₅ P₄ P₃ P₂ P₁ P₀

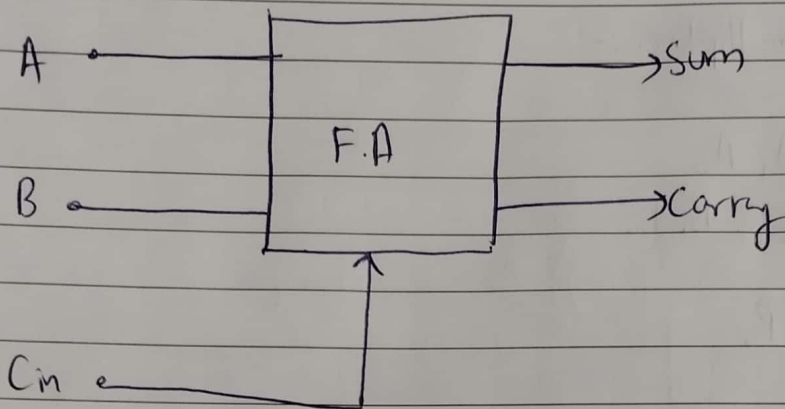
Multiplication:-

$$\begin{array}{r}
 \begin{array}{cccccc}
 & & B_3 & B_2 & B_1 & B_0 \\
 & & \times & & A_1 & A_0 \\
 \hline
 & A_0 B_3 & A_0 B_2 & A_0 B_1 & A_0 B_0 & \\
 + & A_1 B_3 & A_1 B_2 & A_1 B_1 & A_1 B_0 & 0 \\
 \hline
 P_5 & P_4 & P_3 & P_2 & P_1 & P_0
 \end{array} \\
 \uparrow \\
 \text{Carry bit}
 \end{array}$$

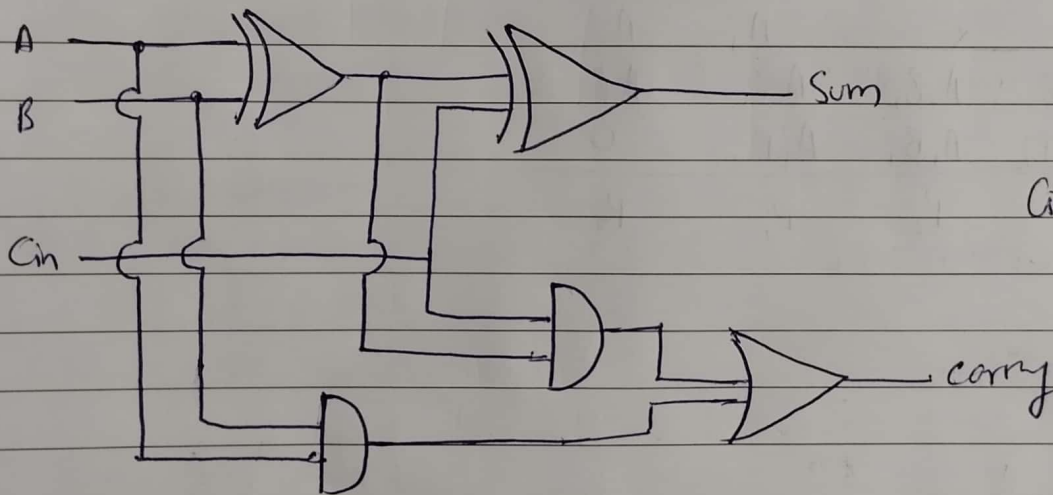
Half adder:-



full-adder



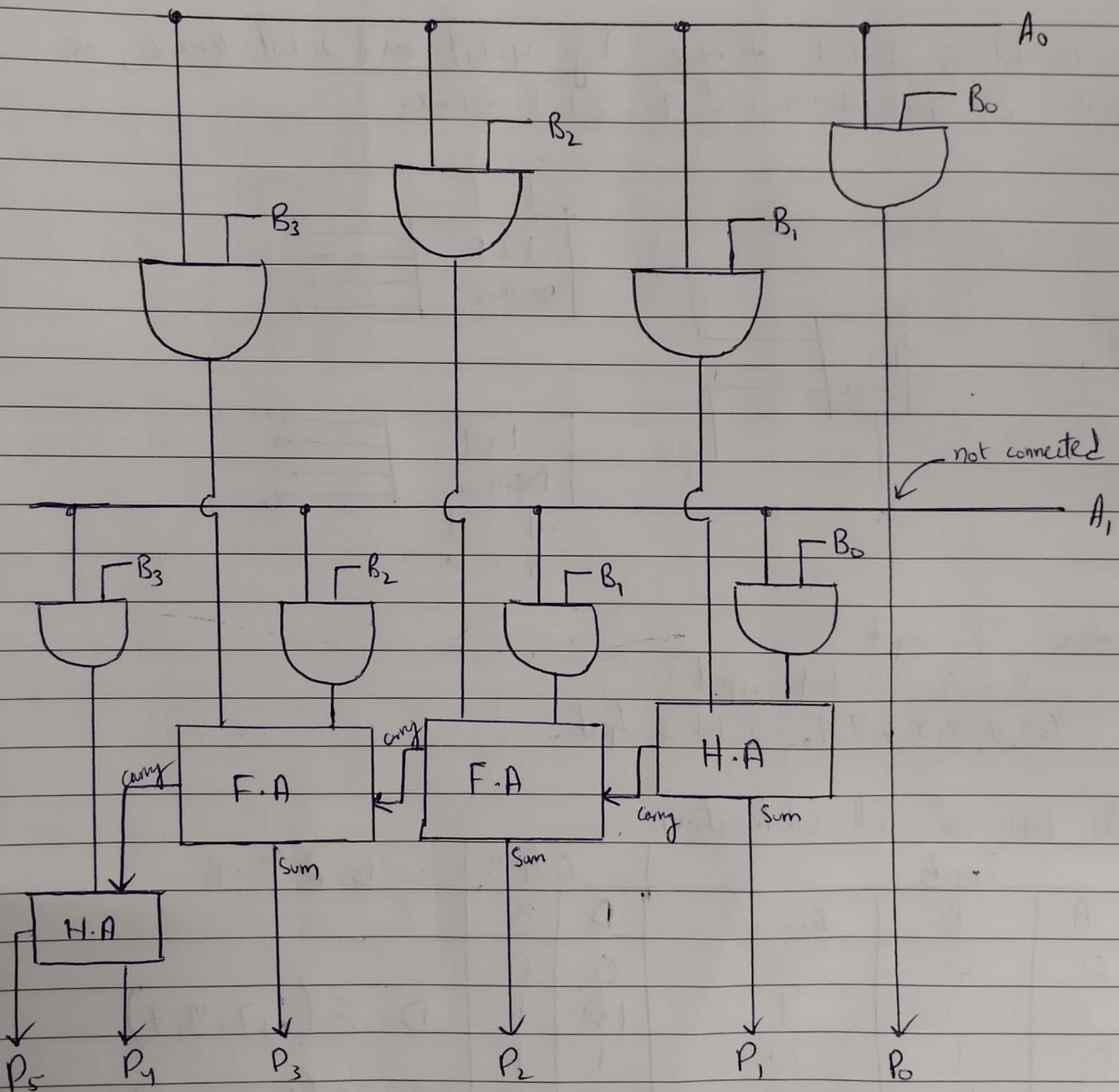
block diagram



Circuit diagram



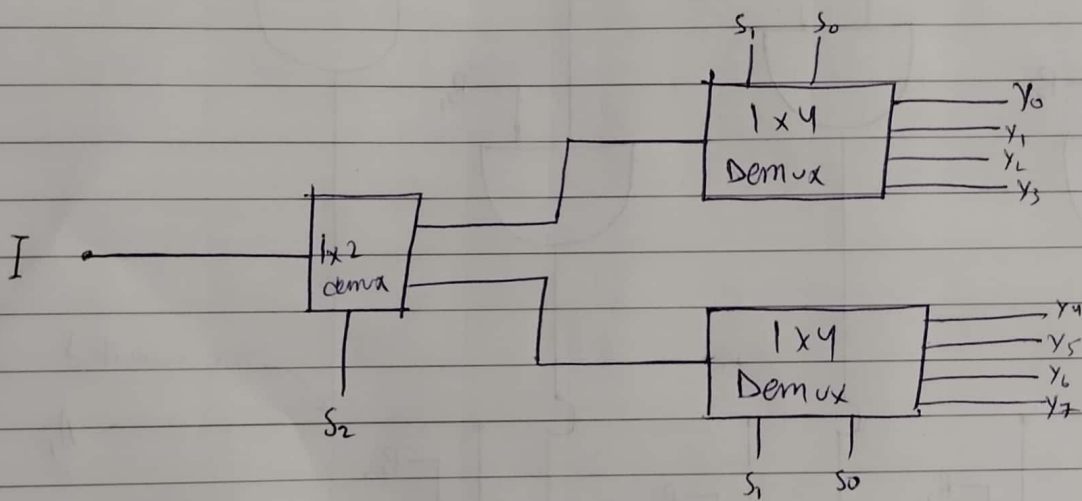
Block diagram of 2-to-4 bit multiplier



b) Implement a full subtractor using 1x4 and 1x2 demux.

- For that we require 3 bit input which is A, B and borrow.
- And we will need a 1x8 bit demux.

To build a 2-bit demux by 4-bit and 2-bit demux, we require 2- 4bit demux and 1- 2bit demux.



Where $I = \text{input}$

$S_2, S_1, S_0 = 3\text{-bit input}$

$Y_7, Y_6, Y_5, Y_4, Y_3, Y_2, Y_1, Y_0 = 8\text{-bit output}$

Truth table of full subtractor:-

Inputs			Outputs	
A	B	Borrow	D	Bi
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

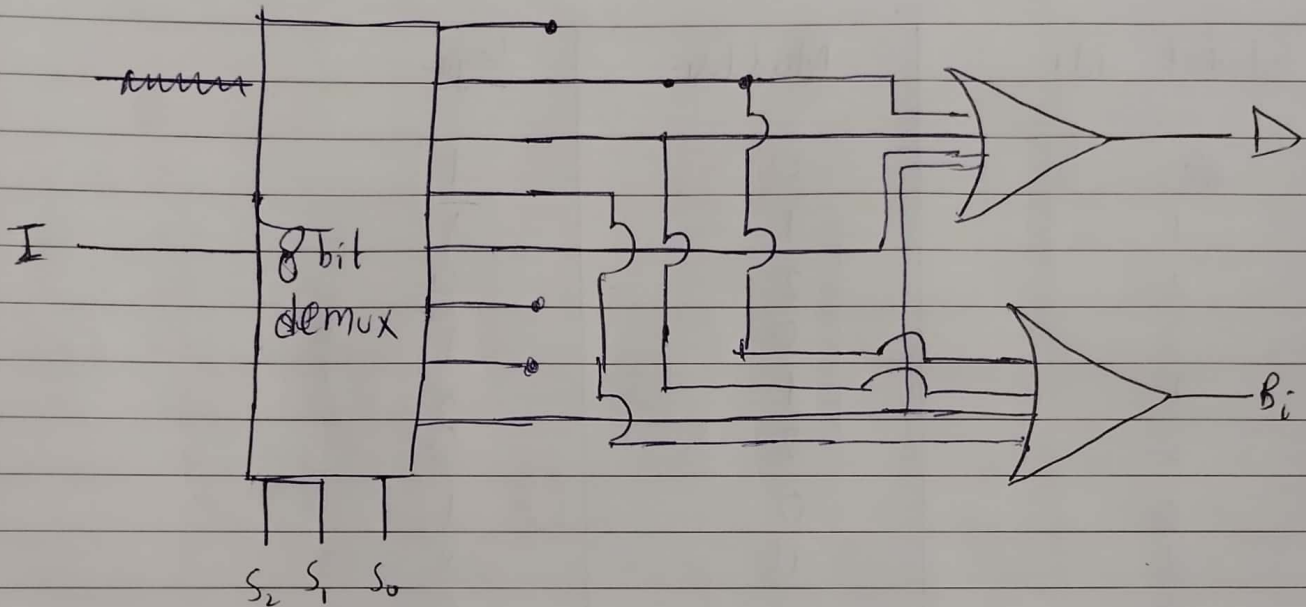
$$D = A - B - B_i$$

$$D = \sum (1, 2, 4, 7)$$

$$B_i = \sum (1, 2, 3, 7)$$



Circuit of full subtraction-



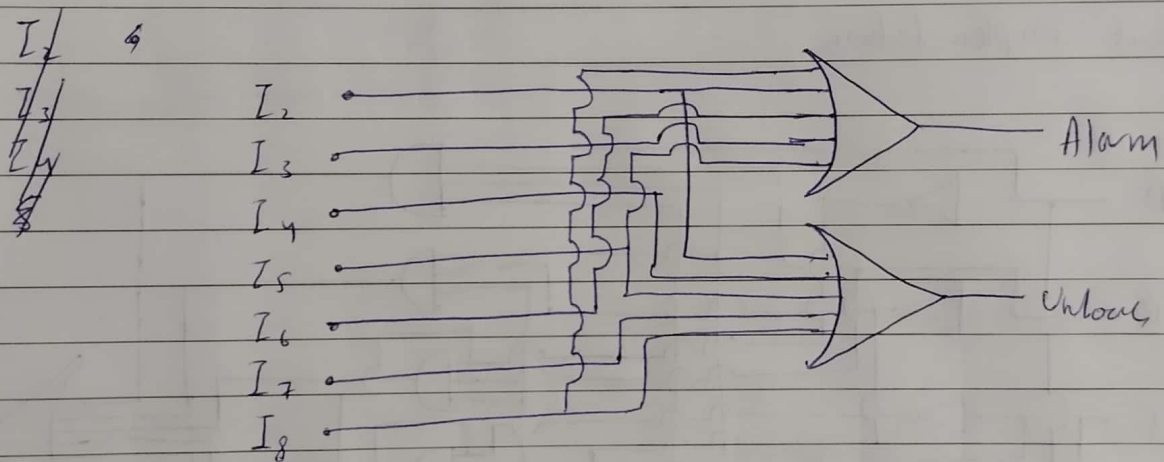
Q61-

a)

$$\text{Unlock} = I_2 + I_4 + I_5 + I_7 + I_8$$

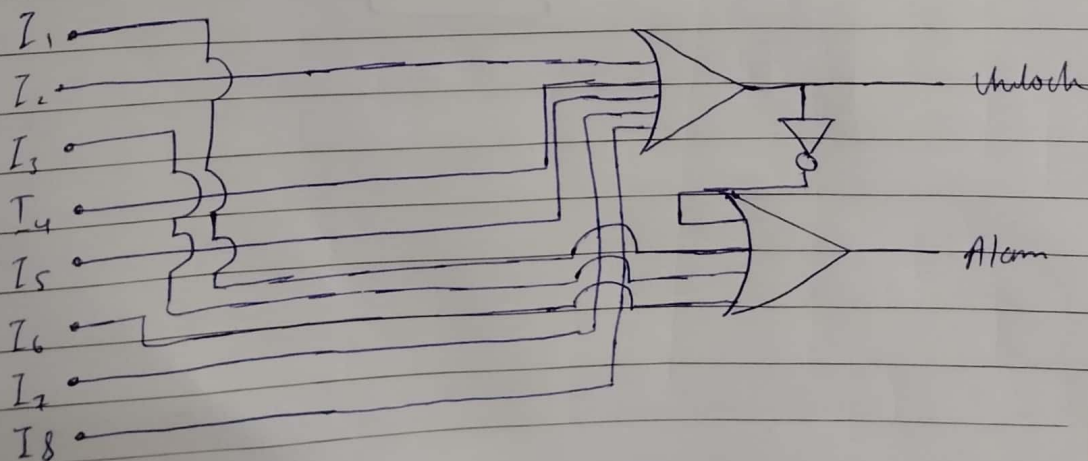
$$\text{Alarm} = I_2 + I_3 + I_5 + I_6 + I_8$$

logic circuit-



- b) The fault is that when the unlock is '1' alarm must be zero, because the door get unlocked, and when the unlock is '0' alarm must be '1'.

$$\text{Alarm} = I_1 + I_3 + I_6$$



$$A'B' + AB$$

$$00 \quad 11 = 1$$

Date _____ 20____

New Truth table:-

I_1	I_2	I_3	I_4	I_5	I_6	I_7	I_8	Unlock	Alarm
1	0	0	0	0	0	0	0	0	1
0	1	0	0	0	0	0	0	1	0
0	0	1	0	0	0	0	0	0	1
0	0	0	1	0	0	0	0	1	0
0	0	0	0	1	0	0	0	1	0
0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	0	1	0	1	0
0	0	0	0	0	0	0	1	1	0

equations, $Unlock = I_2 + I_4 + I_5 + I_7 + I_8$

$Alarm = I_1 + I_3 + I_6$

Q7:-

a) Truth table:-

Student id:-	Modulus-	logic:-
0	0	1
1	1	X
2	2	0
3	0	1
4	1	X
5	2	0
6	0	1
7	1	X
8	2	0
9	0	1
10	1	X
11	2	0
12	0	1
13	1	X
14	2	0
15	0	1

Kuphayan



Q7 b)

- To check which number is bigger, we need to compare each bit,

$$A = A_3 A_2 A_1 A_0 \quad B = B_3 B_2 B_1 B_0$$

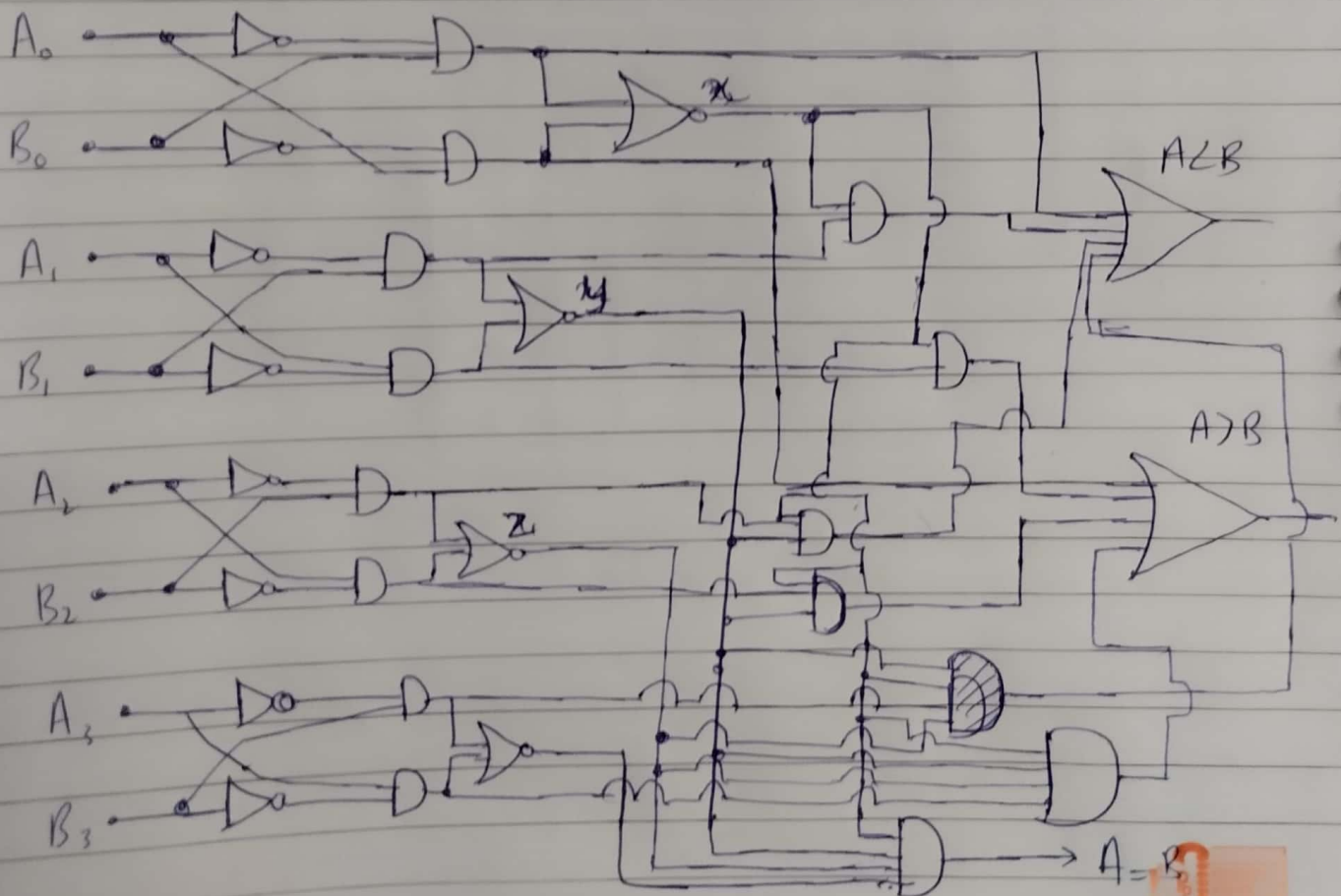
To compare each bit-

$$= A_3 B_3' + x A_2 B_2' + xy A_1 B_1' + xyz A_0 B_0'$$

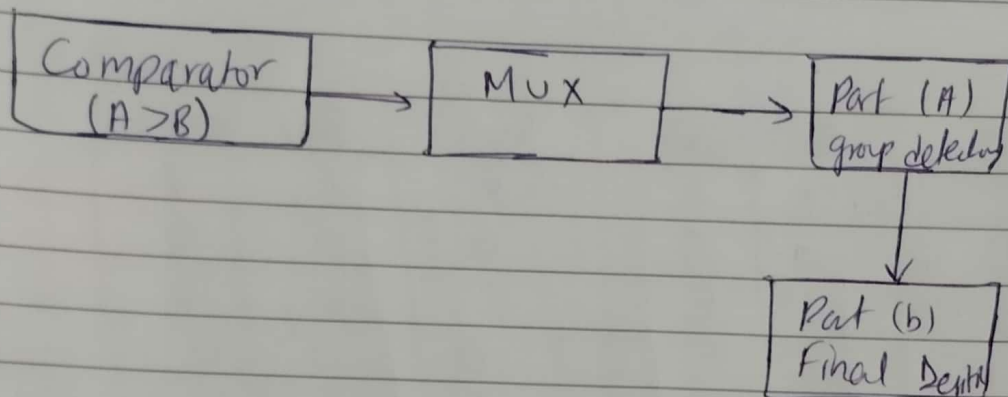
$$x = A_3 \text{ XNOR } B_3$$

$$y = A_2 \text{ XNOR } B_2$$

$$z = A_1 \text{ XNOR } B_1$$

Comparator :-

c)



d)

- Instead of comparing only two IDs it should compare more than that by tree method, it will help it to deal with greater number of people.
- We can improve efficiency by using priority encoders which rank according to highest ID values. Make it more efficient.
- If implementation is hardware, FPGA can be used for dynamic comparison and input handling.
- Implement a sorting network where multiple IDs are stored at once and it sorts according to the highest ID, which will improve the efficiency of logic.
- MUX could be used to select the values of IDs with highest precedence.